ZAALIMA DEVELOPMENT

Customer Churn Prediction & LTV Engine

Week 3 • Day 4

FastAPI — Single Customer Prediction Endpoint

Build and test the POST /predict endpoint for real-time churn prediction

■ Day Objectives

- ✥ Create a Pydantic request model for customer data
- ✥ Build the POST /predict endpoint
- ✥ Apply feature engineering and preprocessing inside the endpoint
- ✥ Return churn probability, prediction, LTV estimate, and SHAP explanation
- ✥ Test the endpoint with curl and the Swagger UI

■ 1. Pydantic Request/Response Models

Pydantic is FastAPI's data validation library. We define the shape of API requests as Python classes — FastAPI automatically validates incoming JSON against these classes and returns helpful error messages for invalid input. This is far better than manually checking every field.

■■ 2. Prediction Endpoint

■ src/api/schemas.py — request and response models

```
from pydantic import BaseModel, Field
from typing import Optional

class CustomerInput(BaseModel):
```

```
    """Input schema for a single customer prediction request."""
    gender: str = Field(..., example='Male')
    SeniorCitizen: int = Field(..., ge=0, le=1, example=0)
    Partner: str = Field(..., example='Yes')
    Dependents: str = Field(..., example='No')
    tenure: int = Field(..., ge=0, example=24)
    PhoneService: str = Field(..., example='Yes')
    MultipleLines: str = Field(..., example='No')
    InternetService: str = Field(..., example='Fiber optic')
    OnlineSecurity: str = Field(..., example='No')
    OnlineBackup: str = Field(..., example='Yes')
    DeviceProtection: str = Field(..., example='No')
    TechSupport: str = Field(..., example='No')
    StreamingTV: str = Field(..., example='Yes')
    StreamingMovies: str = Field(..., example='Yes')
    Contract: str = Field(..., example='Month-to-month')
    PaperlessBilling: str = Field(..., example='Yes')
    PaymentMethod: str = Field(..., example='Electronic check')
    MonthlyCharges: float = Field(..., gt=0, example=70.35)
    TotalCharges: float = Field(..., ge=0, example=1687.65)

class ChurnPredictionResponse(BaseModel):
    customer_id: Optional[str]
    churn_probability: float
    churn_prediction: str          # 'Will Churn' or 'Will Stay'
    risk_level: str               # 'High', 'Medium', 'Low'
    predictive_ltv: float
    ltv_segment: str
    top_risk_factors: list
    recommendation: str
```

■ Add /predict endpoint to src/api/main.py

```
# Add these imports at top of main.py:
from src.api.schemas import CustomerInput, ChurnPredictionResponse
from src.features.engineering import create_all_features
import shap

# Add this endpoint function:
@app.post('/predict', response_model=ChurnPredictionResponse)
async def predict_churn(customer: CustomerInput):
    """
    Predict churn probability and LTV for a single customer.
    Returns risk level, prediction, and top risk factors.
    """
    try:
        # Convert input to DataFrame
        raw = customer.dict()
        # Map Yes/No to 1/0 for binary fields
        binary_map = {'Yes':1,'No':0}
        for col in ['Partner','Dependents','PhoneService','PaperlessBilling']:
            raw[col.lower()] = binary_map.get(raw.pop(col, raw.get(col.lower(),'No')),'N
o')

        df = pd.DataFrame([raw])
```

```
        df.columns = [c.lower() for c in df.columns]
        df['totalcharges'] = pd.to_numeric(df['totalcharges'], errors='coerce').fillna(d
f['monthlycharges'])

        # Apply feature engineering
        df_eng = create_all_features(df)

        # One-hot encode
        ohe_cols = ['internetservice','contract','paymentmethod','multiplelines']
        df_enc = pd.get_dummies(df_eng, columns=[c for c in ohe_cols if c in df_eng.colu
mns])

        # Align columns with training features
        for col in feature_cols:
            if col not in df_enc.columns:
                df_enc[col] = 0
        X = df_enc[feature_cols]

        # Predict
        churn_prob = float(churn_model.predict_proba(X)[:, 1][0])
        churn_flag = 'Will Churn' if churn_prob >= threshold else 'Will Stay'
        risk = 'High' if churn_prob > 0.7 else ('Medium' if churn_prob > 0.4 else 'Low')

        # LTV
        ltv_feat_cols = joblib.load('models/ltv_feature_columns.pkl')
        X_ltv = df_enc.copy()
        for col in ltv_feat_cols:
            if col not in X_ltv.columns: X_ltv[col] = 0
        ltv = float(ltv_model.predict(X_ltv[ltv_feat_cols])[0])
        ltv_seg = 'High Value' if ltv > 2000 else ('Medium Value' if ltv > 800 else 'Low
 Value')

        # SHAP (top 3 risk factors)
        explainer = joblib.load('models/shap_explainer.pkl')
        sv = explainer.shap_values(X)[0]
        top3_idx = np.argsort(np.abs(sv))[-3:][::-1]
        top3 = [f'{feature_cols[i]}: {sv[i]:+.3f}' for i in top3_idx]

        # Recommendation
        if churn_flag == 'Will Churn':
            rec = 'Offer annual contract discount + bundle security services'
        else:
            rec = 'Maintain engagement — consider upsell to premium bundle'

        return ChurnPredictionResponse(
            customer_id=None,
            churn_probability=round(churn_prob, 4),
            churn_prediction=churn_flag,
            risk_level=risk,
            predictive_ltv=round(ltv, 2),
            ltv_segment=ltv_seg,
            top_risk_factors=top3,
            recommendation=rec
        )
```

```
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

■ Test with curl

```
curl -X POST http://localhost:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"gender":"Male","SeniorCitizen":0,"Partner":"No","Dependents":"No",
       "tenure":2,"PhoneService":"Yes","MultipleLines":"No",
       "InternetService":"Fiber optic","OnlineSecurity":"No",
       "OnlineBackup":"No","DeviceProtection":"No","TechSupport":"No",
       "StreamingTV":"No","StreamingMovies":"No",
       "Contract":"Month-to-month","PaperlessBilling":"Yes",
       "PaymentMethod":"Electronic check",
       "MonthlyCharges":70.35,"TotalCharges":140.70}'
```

■ 3. Commit & Checklist

- ■ POST /predict returns churn_probability, risk_level, predictive_ltv
- ■ Swagger UI test works interactively
- ■ Error handling returns 500 with meaningful message

■ Week3-Day4 Commit

```
git add .
git commit -m "Week3-Day4: FastAPI /predict endpoint — churn + LTV + SHAP response"
git push origin main
```

■ Estimated Time: 6 – 8 hours